Defiende el servidor

Taller evaluable de MLOps · 2026-II · Universidad Externado de Colombia
Daniela González Ortiz (trabajo individual) · Grupo A · github.com/danielaestudio02-del/Taller_1

> **La idea de este trabajo.** El reto era clasificar conexiones de red como normales o ataques, pero lo que más me importaba era poder responder la pregunta central del taller: *si pierdo mi carpeta, ¿puedo recuperar una versión concreta del modelo, explicar de qué datos y experimento viene y usarla para predecir?* Para lograrlo, cada pieza apunta a la siguiente con un identificador: **commit de Git → versión de datos en DVC → run de MLflow → revisión en Hugging Face**. Además, cada modelo lleva su huella SHA-256, y el script de predicción se niega a cargarlo si esa huella no coincide.

1. Los datos y cómo los limpié

Usé las particiones publicadas de UNSW-NB15 que pide la guía: `UNSW_NB15_training-set.csv` (175.341 filas) y `UNSW_NB15_testing-set.csv` (82.332 filas). Las descargué el 30 de octubre de 2026 del enlace alternativo del enunciado y verifiqué que su SHA-256 coincidiera con el del anexo (`bec7dd5e…` y `734fe664…`). Lo primero que noté es que los archivos traen un carácter invisible (BOM) al inicio, así que los leí con `utf-8-sig`. No hay valores faltantes.

La variable objetivo es `label`, y tomé el ataque (1) como clase positiva. Saqué tres columnas de las entradas: `id`, porque solo identifica la fila; `attack_cat`, porque dice qué tipo de ataque es y eso sería darle la respuesta al modelo; y obviamente `label`. Quedaron 42 variables: 39 numéricas y 3 categóricas (`proto`, `service` y `state`). Las proporciones de clase son distintas entre archivos: entrenamiento tiene 68 % de ataques y test solo 55 %, algo que después resultó importante.

Lo que más me sorprendió fueron los duplicados. En entrenamiento solo hay 101.040 entradas distintas: el 42 % de las filas repite exactamente las mismas variables de otra fila. Además encontré 229 grupos de filas idénticas con etiquetas contradictorias (940 filas), es decir, la misma conexión marcada a veces como normal y a veces como ataque; en test hay 6 grupos así. **Decidí no borrar nada:** los duplicados reflejan con qué frecuencia aparece cada tipo de tráfico, y las contradicciones son ruido que ningún modelo puede resolver. Lo que sí hice fue asegurarme de que las filas idénticas nunca quedaran repartidas entre entrenamiento y validación.

2. Cómo armé las particiones sin fuga de información

Separé entrenamiento y validación **solo a partir del archivo de entrenamiento**. Usé `StratifiedGroupKFold` con 5 pliegues y semilla 42, y tomé el primer pliegue como validación. Cada "grupo" son las filas con entradas idénticas, de modo que un duplicado no puede estar a la vez en los dos lados; además se mantiene la proporción de ataques. Así quedaron 140.274 filas de entrenamiento y 35.067 de validación, con **cero grupos compartidos**. Las huellas de cada partición están en `auditoria.json`, y comprobé que el proceso es reproducible: al volver a correr la preparación, DVC no detecta ningún cambio.

Todo el preprocesamiento (imputación, codificación y escalado) va dentro de un `Pipeline` de scikit-learn que solo se ajusta con entrenamiento y se guarda junto con el modelo. Al test no lo toqué hasta el final; solo revisé su estructura y los cruces. Ahí apareció algo que hay que tener en cuenta: **el 10,4 % de las filas de test (8.541) son idénticas a alguna de entrenamiento o validación**, y dos valores de `state` (ACC y CLO) no aparecen en entrenamiento. O sea, el test no es del todo independiente.

En DVC guardé dos versiones de los datos: la **v1**, con los CSV tal cual los recibí (huella `fe93051a…`, commit `1b826c1f`), y la **v2**, con los datos preparados y las particiones (huella `53545c81…`, commit `9a17cd22`). El almacenamiento quedó en un repositorio público de Hugging Face (`Daniela201000/defiende-el-servidor-dvc`), así que cualquiera puede recuperarlos con `dvc pull`.

3. Los cuatro experimentos

Probé dos configuraciones de cada familia, todas con las mismas particiones y la misma semilla. Para que cada resultado quedara amarrado a un código exacto, el script de entrenamiento no corre si hay cambios sin commit; los cuatro se entrenaron en el commit `9a17cd22`. En MLflow quedó todo registrado: la hipótesis, el cambio, los parámetros, el commit, la versión de datos, las métricas, la matriz de confusión y el modelo completo con su huella. Si alguna métrica tuviera denominador cero la reportaría como 0, aunque no pasó en ningún caso.

Experimento · run	Qué probé y por qué	Recall	Precision	FPR	Macro-F1
exp1 · `d918b875`	**Lineal.** Regresión logística básica (one-hot + escalado) como punto de partida.	0,9886	0,9223	0,1776	0,9225
exp2 · `420be7f7`	**Lineal.** Logaritmo en las variables numéricas y más regularización, porque los bytes y las tasas tienen valores extremos.	0,9771	0,9354	0,1437	0,9274
exp3 · `831fc6f5`	**Árboles.** Random Forest, para captar relaciones no lineales entre TTL, estado y conteos.	0,9808	0,9546	0,0994	0,9476
exp4 · `a5ca5470`	**Árboles.** HistGradientBoosting, que va corrigiendo los errores de los árboles anteriores.	0,9764	0,9611	0,0842	0,9502

Las hipótesis se cumplieron en buena medida. El logaritmo sí ayudó al modelo lineal (la FPR bajó de 0,178 a 0,144), pero no alcanza: un modelo lineal sigue generando demasiadas falsas alarmas. Con los árboles la FPR bajó a menos de la mitad sin perder mucho recall. El mejor fue exp4, que además pesa solo 1,1 MB, frente a los 44 MB del Random Forest.

4. De v1 a v2: qué gané y qué perdí

	v1	v2
Modelo y umbral	exp1, regresión logística · 0,50	exp4, HistGradientBoosting · 0,65
Commit de la solución	`330015e9da6c114e870f444c720d585952fbfaac`	`2ae6479c9528c7dcc2342908618d4b432c8131fb`
Revisión en Hugging Face	`691479df2d72c01e99bd1b32c9d5b82368ff8b70`	`e2f7bfdd932f383d7cf40c5413d5d407408889ba`
Huella del modelo (SHA-256)	`2b350cdc…a8ab09c6c`	`ac60460c…69d5f2b62`
Validación: recall / precision / FPR / macro-F1	0,9886 / 0,9223 / 0,1776 / 0,9225	0,9540 / 0,9760 / 0,0499 / 0,9463
Matriz de validación (TN FP / FN TP)	9.211 1.989 / 272 23.595	10.641 559 / 1.098 22.769

v1 fue mi primera solución funcional. Para v2 cambié dos cosas: **la familia de modelo y el umbral**. **Lo que gané:** las falsas alarmas sobre tráfico normal bajaron de 1.989 a 559, es decir, la FPR pasó de 17,8 % a 5,0 %, y subieron la precisión y el macro-F1. **Lo que perdí:** el recall bajó de 0,989 a 0,954, que son 826 ataques más que se escapan. Revisando por tipo de ataque (solo como análisis, porque `attack_cat` no entra al modelo), vi que v2 detecta el 73 % de los *Fuzzers* frente al 95 % de v1, y el 87 % de los de tipo *Analysis* frente al 99 %. En DoS, Exploits, Generic, Reconnaissance, Backdoor y Worms se mantiene igual o mejora.

¿Por qué acepto esos errores? Con v1, una de cada cinco conexiones normales dispara una alarma. En la práctica, un equipo de seguridad que recibe tantas alertas falsas termina ignorándolas, y eso también es peligroso. Prefiero un modelo que alerta menos pero con más confianza. Los Fuzzers son el precio de esa decisión, porque se parecen mucho al tráfico normal.

5. Cómo elegí el umbral (sin mirar el test)

Escribí la regla **antes** de evaluar el test y la dejé guardada en `configs/selection.yaml`. Primero, elegir el modelo con mejor macro-F1 en validación, que fue exp4. Después, entre los umbrales de 0,05 a 0,95 (de 0,01 en 0,01), quedarme con el que detecte más ataques sin pasar de un 5 % de falsas alarmas. Ese umbral fue **0,65**, y toda la curva está en `seleccion_umbral.csv`. La regla final es: si `score_attack ≥ 0,65`, es ataque.

Para demostrar que no hice trampa, la selección quedó guardada en el commit `2ae6479c`, que es **anterior** al commit de la evaluación en test (`701a209c`). Además, el script de evaluación no deja correr el test si la selección no está guardada primero en Git.

6. El test final

Filas	Recall	Precision	FPR	Macro-F1	Accuracy	TN · FP · FN · TP
82.332	0,9701	0,8681	0,1806	0,8996	0,9024	30.319 · 6.681 · 1.357 · 43.975

Evalué v2 una sola vez, sobre todo el archivo de test y sin quitar ninguna fila. Las predicciones de cada fila están en `predicciones-test.csv`, y las métricas se pueden recalcular con `recompute_metrics.py`; dan exactamente lo mismo. **El resultado fue peor de lo que esperaba, y lo dejo tal cual.** El recall se mantuvo bien (0,970), pero la FPR subió de 5 % en validación a 18 % en test. Mi explicación es que el tráfico normal del archivo de test es diferente al de entrenamiento, algo que ya se ha reportado con estas particiones de UNSW-NB15. Además, test tiene menos ataques en proporción, y eso baja la precisión. Podría haber movido el umbral para que el número se viera mejor, pero eso sería usar el test para decidir, y entonces dejaría de ser una evaluación honesta. Si hiciera una v3, recalibraría el umbral con datos normales más representativos.

7. La prueba de recuperación

Esta era la parte más importante. Con `scripts/recuperacion.sh` simulé que había perdido todo: el script clona el repositorio en una carpeta nueva, crea un entorno nuevo, instala las dependencias y usa cachés vacías tanto para DVC como para Hugging Face. Después baja los datos con `dvc pull`, verifica sus huellas, descarga v1 y v2 de Hugging Face usando su SHA completo, predice sobre 25 filas fijas de validación y compara con las predicciones que había guardado antes de publicar. **Resultado (30-10-2026, commit `3da339d1`): todo coincidió.** `dvc pull` trajo los datos con las huellas correctas; las 25 etiquetas de v1 y de v2 fueron idénticas, y la diferencia máxima en las probabilidades fue de 3,7e-11 y 4,9e-11, muy por debajo de la tolerancia que fijé (1e-9). Las huellas de los modelos descargados también coincidieron con las registradas. El log completo está en `evidencias/recuperacion/`.

8. Limitaciones

- UNSW-NB15 es de 2015 y se generó en un laboratorio, así que esto no demuestra que el modelo funcione en una red real de hoy. De hecho, lo que pasó con la FPR en test muestra que el umbral no se traslada bien a otro tráfico.
- Como el 10,4 % del test ya estaba en entrenamiento o validación, el resultado de test es algo optimista. Y las etiquetas contradictorias ponen un límite al error mínimo posible.
- v2 falla más con los Fuzzers. El umbral busca el mejor recall con un tope de falsas alarmas, pero no tiene en cuenta que algunos errores pueden costar más que otros.
- MLflow quedó en local y lo entrego exportado. Los modelos se guardan con pickle, así que solo se deben cargar versiones verificadas por su huella, y eso es justo lo que hace `predict.py`.

Referencias. Moustafa, N. y Slay, J. (2015). UNSW-NB15: a comprehensive data set for network intrusion detection systems. MilCIS · research.unsw.edu.au/projects/unsw-nb15-dataset · Documentación de scikit-learn (common pitfalls), DVC, MLflow Tracking y Hugging Face Hub (descarga por revisión y model cards). **Uso de IA:** usé Claude Code (Anthropic) como apoyo para el código, la ejecución y la redacción; los detalles están en `entrega.md`.